Q1. What are the most common reasons for missing data in ETL pipelines?

Answer :- Missing data in ETL (Extract, Transform, Load) pipelines is rarely a single-point failure; it typically stems from issues at the source, during the journey, or due to the logic applied to the data.

The following are the most common reasons for missing data, categorized by where they occur in the pipeline:

1. Source System Issues

The pipeline can only move what is available. If the data isn't at the source, it won't reach the destination.

- **Optional Fields:** Users may skip non-mandatory fields in a UI (e.g., an optional "Middle Name" or "Referral Code").
- **System Glitches:** Bugs in the source application might fail to save certain attributes to the database.
- **Human Error:** Manual data entry often leads to omissions or "dummy" values (like `N/A` or `0`) that are later filtered out or treated as nulls.

2. Extraction Failures

Issues that occur while the ETL tool is trying to pull data from the source.

- **API Timeouts:** If an API call times out before the full payload is delivered, the pipeline might process a partial (and thus "missing") dataset.
- **Schema Drift:** If a source database renames a column or changes its structure, the extraction script may fail to find the field, leading to null values in the target.
- **Connectivity Drops:** Intermittent network issues during a large file transfer (like SFTP) can result in truncated files.

3. Transformation Logic Errors

Data often goes "missing" because the transformation rules unintentionally exclude it.

- **Inner Join Drop-offs:** If you perform an `INNER JOIN` between two tables and a record in Table A doesn't have a matching key in Table B, that record is dropped entirely from the result.
- **Data Type Mismatches:** If a pipeline tries to force a string like "Unknown" into a numeric column, the ETL tool may default that value to `NULL` to prevent the whole job from failing.
- **Aggressive Filtering:** Hardcoded filters (e.g., `WHERE status = 'active'`) may exclude data that should have been captured but didn't meet a strict criteria.

4. Loading & Environment Issues

Even after successful transformation, the data can fail to "land" in the final destination.

- **Partial Loads:** If a load job crashes halfway through (e.g., due to disk space or a lost connection), only a portion of the data will exist in the target table.
- **Constraint Violations:** If a target table has a `NOT NULL` constraint and the incoming data has a null, the entire row (or even the entire batch) might be rejected.
- **Primary Key Collisions:** If the pipeline tries to load a duplicate record and the target system is set to "ignore duplicates," the new data appears to be missing.

Q2. Why is blindly deleting rows with missing values considered a bad practice in ETL?

Answer :- Blindly deleting rows with missing values (often called **Listwise Deletion**) is usually the "easiest" technical solution, but it is considered a dangerous practice in ETL because it often creates more problems than it solves.

Here are the four primary reasons why this is discouraged:

1. Introduction of Systematic Bias[2]

The biggest risk is that data is rarely missing at random.[3] If a specific group of people or a certain type of sensor is more likely to have missing values, deleting those rows removes that entire segment from your analysis.

- **Example:** In a customer survey, if high-income individuals are less likely to report their salary, deleting rows with missing "income" values will lead you to calculate an average income that is **lower** than the actual truth.[4] You haven't just cleaned the data; you've biased it.[5]

2. Loss of Statistical Power

ETL pipelines often serve Machine Learning models or complex analytics.[6] Deleting rows reduces your sample size ([7]n).[8]

- **The Impact:** Smaller datasets make it harder to detect subtle patterns or trends.[9] If your dataset has 10 columns and 5% of the data is missing randomly across different columns, you could end up deleting up to 50% of your total rows, even though most of the data in those rows was perfectly valid.

3. Destruction of "Informative Missingness"

In many systems, the fact that a value is missing is **data in itself**.[10] Deleting the row throws away this signal.[11]

- **Example:** In a medical ETL pipeline, a missing "Lab Result" might mean the test was never ordered because the patient was healthy. In a credit application, a missing "Middle Name" might correlate with specific nationalities.
- **Alternative:** Instead of deleting, engineers often use a **Missingness Flag** (a boolean column) to tell the model that the value was absent, which can be a powerful predictive feature.[12]

4. Violation of Referential Integrity

If you delete a row from a "Sales" table because the "Tax Amount" was missing, you may create "orphan" records in related tables or break your financial totals.

- **Example:** If you delete 100 sales records due to missing metadata, your ETL pipeline will report a lower "Total Revenue" than what actually hit the bank account, leading to reconciliation failures and a loss of trust in the data platform.

Q3. Explain the difference between:

- Listwise deletion
- Column deletion

Answer :- **1. Listwise Deletion (Row Deletion)**

Listwise deletion is the process of removing an **entire record (row)** from the dataset if it is missing a value in any of the columns required for analysis.

- **How it works:** If you have 100 columns and just one is `NULL`, the entire row is purged.
- **The Goal:** To ensure the final dataset is "complete" so that downstream models or functions don't crash when encountering a null.
- **Appropriate Scenario:** When the missing value is in a **Primary Key** or a **Critical Join Key**. For example, if you are processing "Sales Orders" and the `Order_ID` is missing, you cannot link that sale to a customer or a product. Since the record is functionally useless for any relational analysis, deleting the row is the safest way to maintain data integrity.

2. Column Deletion (Feature Dropping)

Column deletion involves removing an **entire attribute (column)** from the dataset for every single record, regardless of whether some rows have data for it.

- **How it works:** If a specific field is consistently empty or unreliable, you drop that field entirely from the ETL schema.
- **The Goal:** To simplify the dataset and remove "noise" that could mislead a model or waste storage/processing power.
- **Appropriate Scenario:** When a column has a **high percentage of missing values** (typically >50–70%) and is not essential. For example, if an e-commerce site recently added a "Secondary Phone Number" field and only 2% of users have filled it out, that column is likely too sparse to provide any statistical value. Dropping it reduces the "width" of your data and speeds up the pipeline.

Q4. Why is median imputation preferred over mean imputation for skewed data such as income?

Answer :- When data is **skewed**, like income or house prices, the "average" (mean) is no longer a reliable representation of the "typical" value. In these cases, **median imputation** is preferred because it is **robust to outliers**.

Here is the breakdown of why the mean fails and the median succeeds in a skewed ETL pipeline:

1. The "Bill Gates" Effect (Outlier Sensitivity)

In a skewed distribution, a few extreme values (outliers) pull the mean toward the "tail" of the distribution.

- **The Problem:** If you have 10 people earning $50k and one person earning $10 million, the **mean** income is nearly $1 million.
- **The Consequence:** If you use that mean ($1M) to fill in missing values for an average person, you are imputing a value that is 20 times higher than their likely reality. This "pollutes" your dataset with unrealistic values.

2. Resistance to Skewness

The **median** is the middle-most value. It doesn't care if the highest value is $100k or $100 million; it only cares that it is the "highest."

- **The Benefit:** In the same example above, the median remains $50k.
- **The Result:** When you impute the median, you are filling the gap with a value that actually represents the 50th percentile of your population, keeping the "center of mass" of your data intact.

3. Preserving the Data Distribution

Mean imputation on skewed data tends to create a massive "spike" in a location where very few real data points actually exist.

- **Mean Imputation:** Distorts the distribution by moving the average of the imputed records toward the extreme tail.
- **Median Imputation:** Better preserves the central tendency, ensuring that downstream Machine Learning models or BI reports aren't misled by artificially inflated averages.

Q5. What is forward fill and in what type of dataset is it most useful?

Answer :- **Forward Fill** (often abbreviated as **ffill**) is a data imputation technique where a missing value is replaced by the **last observed (non-null) value** from the preceding row.

In simple terms, you "carry forward" the most recent known piece of information until a new value is recorded.

How it Works

Imagine a dataset tracking the temperature of a server room every hour:

Time	Temperature	Action (Forward Fill)
09:00	22°C	Observed
10:00	NULL	**22°C** (Carried from 09:00)
11:00	NULL	**22°C** (Carried from 10:00)
12:00	25°C	New observation

When is Forward Fill most useful?

Forward fill is specifically designed for **Time-Series Data** where observations are continuous and the state of a variable is unlikely to change drastically between short intervals.

It is most effective in these three scenarios:

1. Low-Volatility Time Series

Useful when the data represents a state that stays constant until an event occurs.[3]

- **Example:** A bank balance. If you don't make a transaction on Tuesday, your balance is exactly the same as it was on Monday. Forward-filling Monday's balance into Tuesday is logically sound.

2. IoT and Sensor Data

In many sensor systems, a device only transmits a message when the value *changes* to save battery or bandwidth (this is called "Report by Exception").

- **Example:** A smart thermostat might only send a signal when the temperature moves by 0.5 degrees. If the 10:00 AM reading is missing, it is highly probable that the temperature is still what it was at 9:55 AM.

3. Financial Market Data

When working with stock prices, especially in "tick data" where trades happen at irregular intervals.

- **Example:** If no trade happened at 10:01:05, the "current price" of the stock is simply the price of the last executed trade at 10:01:04.

Q6. Why should flagging missing values be done before imputation in an ETL workflow?

Answer :- **1. Preserving the "Signal" of Missingness**

As discussed earlier, the fact that data is missing is often a piece of data itself. Once you impute a value (e.g., filling a null with the median of **$50,000**), the information that the value was originally missing is **permanently lost**.

- **The Benefit:** A machine learning model can use the flag to learn that "people who don't report income behave differently than those who do." If you only provide the imputed $50,000, the model treats that person exactly like someone who actually earned $50,000, which might be incorrect.

2. Distinguishing "Real" vs. "Synthesized" Data

In a production ETL pipeline, transparency is key. If a data analyst looks at the final table and sees a column full of values, they might assume the source system is working perfectly.

- **The Benefit:** By keeping a flag, you allow downstream users to calculate the **Data Quality Score**. They can see that while the `price` column is 100% full, 30% of those values were "synthesized" via imputation. This prevents false confidence in the data's accuracy.

3. Impact on Statistical Uncertainty

Imputation, by definition, is a guess. Whether you use a mean, median, or a complex regression, you are introducing "artificial" data into the system.

- **The Benefit:** Flags allow statisticians to account for the added uncertainty. When calculating margins of error, they can treat imputed values differently than observed values, leading to more honest and rigorous reporting.

The ETL Workflow: Best Practice Order

1. **Detection:** Identify where the `NULL` values exist.
2. **Flagging (The Missing Step):** Create a binary column (`0` or `1`) indicating where the data was missing.
3. **Imputation:** Fill the original `NULL` values using your chosen strategy (Mean, Median, ffill, etc.).
4. **Validation:** Ensure the flags match the locations of the new imputed values.

Q7. Consider a scenario where income is missing for many customers.

Answer :- ### 1. Identifying "Privacy-Conscious" or High-Value Segments

There is often a strong correlation between high net worth and data privacy.

- **The Insight:** If your ETL flags show that customers with missing income also have high transaction volumes but low engagement with marketing emails, they likely represent a **"Silent Wealth"** segment.
- **Business Action:** Instead of aggressive mass-marketing, target this group with "exclusive" or "concierge" services that emphasize privacy and high-end benefits.

2. Spotting Friction in the Customer Journey

If income is missing for a specific cohort (e.g., users who signed up via a mobile app vs. desktop), it points to a **User Experience (UX) failure**.

- **The Insight:** High missingness on mobile might mean the "Income" dropdown is difficult to use on small screens or that the step is too intrusive during a quick mobile signup.
- **Business Action:** Streamline the onboarding process or move the income question to a later "profile completion" stage to reduce churn.

3. Segmenting by Employment Type

Sometimes, customers leave income blank because they don't have a "standard" monthly salary to report.

- **The Insight:** Missing income can be a proxy for **Gig Economy workers, students, or retirees** whose income is non-linear or comes from multiple sources.
- **Business Action:** Develop specialized financial products (like flexible credit lines) tailored for people with non-traditional income streams.

4. Detecting "Trust Deficits"

Data missingness is often a mirror of how much a customer trusts a brand.

- **The Insight:** If a new product launch shows a spike in missing demographic data compared to older products, it may indicate that customers don't yet see the **value exchange**. They are asking, *"Why do you need to know how much I make just to sell me this?"*
- **Business Action:** Improve the "Why we ask" micro-copy in the UI to explain how providing income helps customize their experience (e.g., "We ask this to pre-approve you for higher credit limits").

Q9 :- Imputation

Handle missing values in Monthly_Sales using:

- Forward Fill

Tasks:

1. Apply forward fill
2. Show before vs after values
3. Explain why forward fill is suitable here

Answer :- 1. Solution In sql file

2.

Customer_ID	Name	Original Sales	Imputed Sales (FFILL)	Source of Imputed Value
101	Rahul Mehta	12000	**12000**	Original Value
102	Anjali Rao	NaN	**12000**	Carried from ID 101
103	Suresh Iyer	15000	**15000**	Original Value
104	Neha Singh	NaN	**15000**	Carried from ID 103
105	Amit Verma	18000	**18000**	Original Value
106	Karan Shah	NaN	**18000**	Carried from ID 105
107	Pooja Das	14000	**14000**	Original Value
108	Riya Kapoor	16000	**16000**	Original Value

3. **Why Forward Fill is suitable here**

Forward Fill is a common imputation technique in data analysis for several reasons:

1. **Preservation of Trend:** If the data is sorted by time or a sequential ID, Forward Fill assumes that the most recent known state is the best predictor for the missing period.
2. **Maintaining Dataset Size:** Unlike dropping rows, which would result in losing **5 records** from this small dataset, imputation allows you to keep all 8 records for analysis.
3. **Simplicity:** It is computationally less expensive than predictive modeling or mean imputation while often being more accurate for sequential or time-series data where "last known value" is a logical placeholder.

Q10. Flagging Missing Data
Create a flag column for missing Income.
Tasks:

1. Create Income_Missing_Flag (0 = present, 1 = missing)
2. Show updated dataset
3. Count how many customers have missing income

Answer :- **Task Summary**

- **Flag Logic:** The `CASE` statement assigns a **1** if the `Income` value is missing (NULL or NaN) and a **0** if the data is present.
- **Total Missing Count:** Looking at the data, **3 customers** (IDs 102, 104, and 107) have missing income values.